

DESARROLLO WEB I

Caso: El "Bug" de 440 Millones de Dólares (Knight Capital Group) En 2012, una de las mayores operadoras de bolsa de EE. UU. perdió 440 millones de dólares en 45 minutos. ¿La razón? Al implementar software nuevo, olvidaron actualizar el código en uno de los ocho servidores. No tenían un control de versiones automatizado y robusto que asegurara que todos los nodos corrieran la misma "rama" de producción.

¿Alguna vez han perdido una funcionalidad que ya operaba por intentar arreglar otra, o han terminado con archivos llamados "index_final_v2_este_si.html"?

¿Qué pasaría si tu equipo de desarrollo web modifica simultáneamente el archivo index.html, alguien rompe el diseño CSS y no existe forma de volver a la versión que sí funcionaba?

- ¿Cómo recuperamos una versión estable?
- ¿Cómo sabemos quién cambió qué?
- ¿Cómo evitamos sobrescribir el trabajo de otro compañero?
- ¿Cómo desarrollamos una nueva funcionalidad sin dañar la versión principal?

Caso de éxito breve

Git se convirtió en el estándar de facto del desarrollo colaborativo moderno, debido a su capacidad para gestionar revisiones, ramas y fusiones de código de forma flexible y robusta en proyectos de software de distinta escala. La literatura científica destaca que los sistemas de control de versiones distribuidos, como Git, son fundamentales para la evolución del software y la coordinación entre desarrolladores (Cesario, Interian y Murta 2017; Khelladi et al. 2021).

Objetivos

- Comprender el concepto de control de versiones.
- Aplicar comandos básicos de Git en entorno local.
- Implementar buenas prácticas de commits.
- Gestionar ramas y realizar merges correctamente.

Concepto de control de versiones

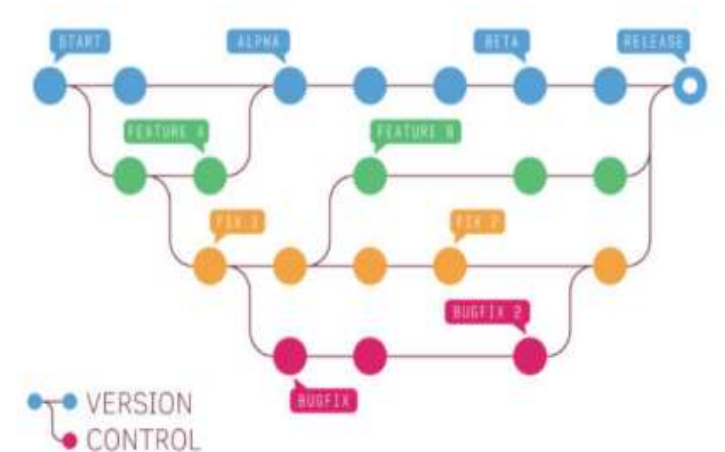
Definición:

El control de versiones es un sistema que permite registrar, rastrear, comparar y recuperar cambios realizados en archivos de un proyecto a lo largo del tiempo.

Según Spinellis (2012), los sistemas de control de versiones permiten gestionar la evolución del software y coordinar el trabajo en equipo de manera eficiente.

Puntos clave:

- ☀ Trazabilidad (quién, cuándo, qué)
- ☀ Reversibilidad.
- ☀ Trabajo en paralelo.



Concepto de control de versiones

¿Por qué es vital en desarrollo web?

En proyectos web trabajamos constantemente con:

- ✓ index.html
- ✓ style.css
- ✓ app.js
- ✓ componentes React/Vite
- ✓ configuraciones
- ✓ assets

Los cambios son frecuentes y a veces riesgosos:

- ➡ Una corrección en CSS rompe el responsive.
- ➡ Una nueva funcionalidad JS genera errores.
- ➡ Una integración de formularios daña otra sección.

Git permite:

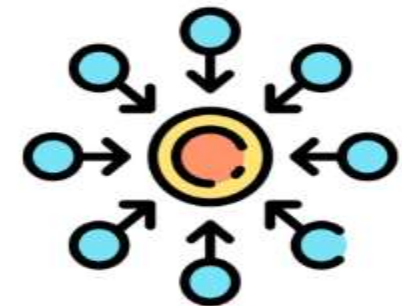
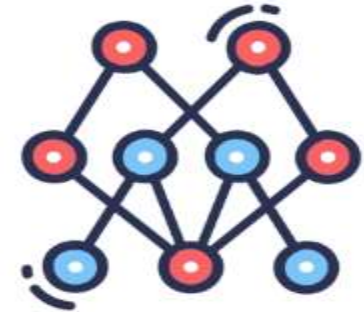
- Trabajar con seguridad.
- Probar sin miedo.
- Colaborar ordenadamente.
- Recuperar versiones estables.

Concepto de control de versiones

Tipos de control de versiones

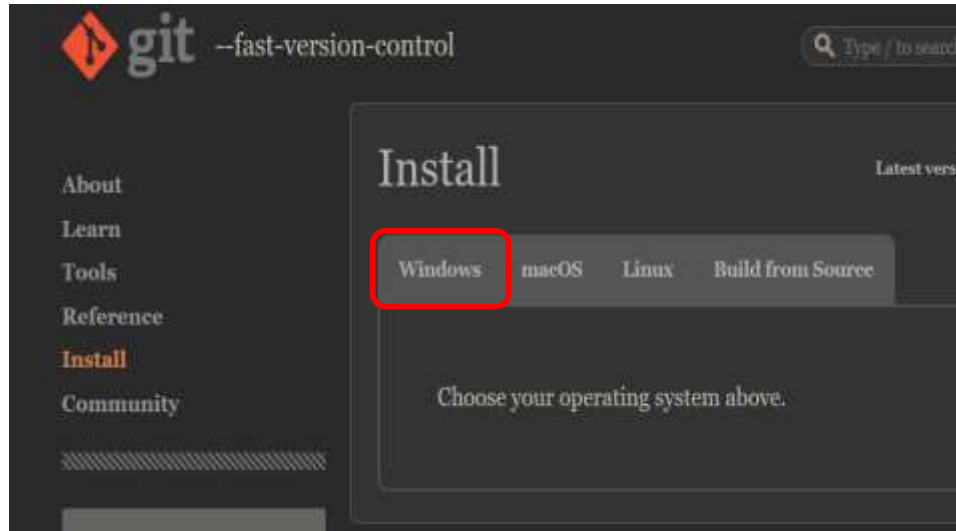
- Local: historial solo en tu máquina.
- Centralizado: servidor central.
- Distribuido: cada desarrollador tiene una copia completa del historial.

Los sistemas de control de versiones son un componente central del desarrollo de software contemporáneo, ya que permiten gestionar revisiones secuenciales y variantes concurrentes (por ejemplo, mediante ramas), facilitando la evolución y mantenimiento del software (Berger et al. 2021). Asimismo, los flujos de trabajo basados en Git son ampliamente adoptados por su flexibilidad y robustez en colaboración (Khelladi et al. 2021).

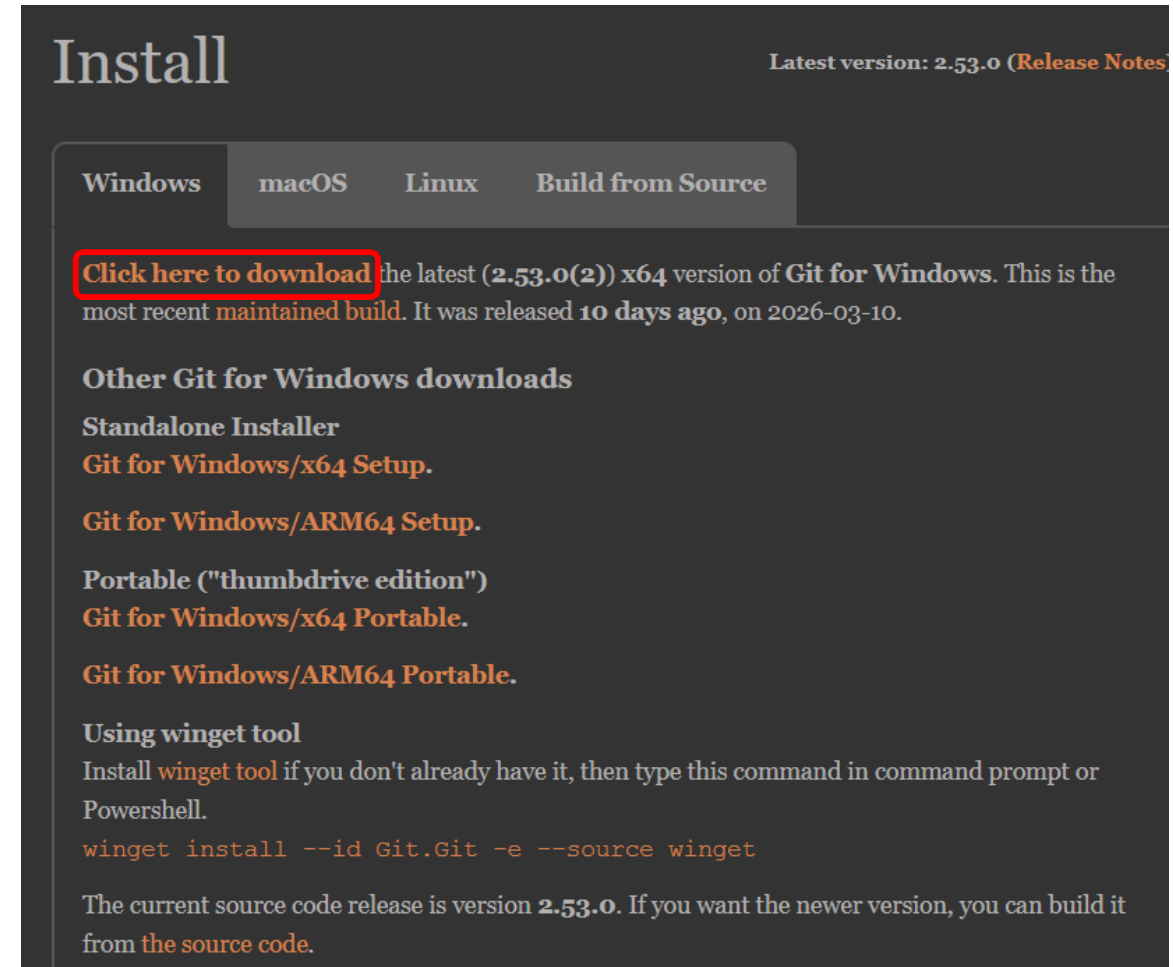


Instalación y configuración de GIT

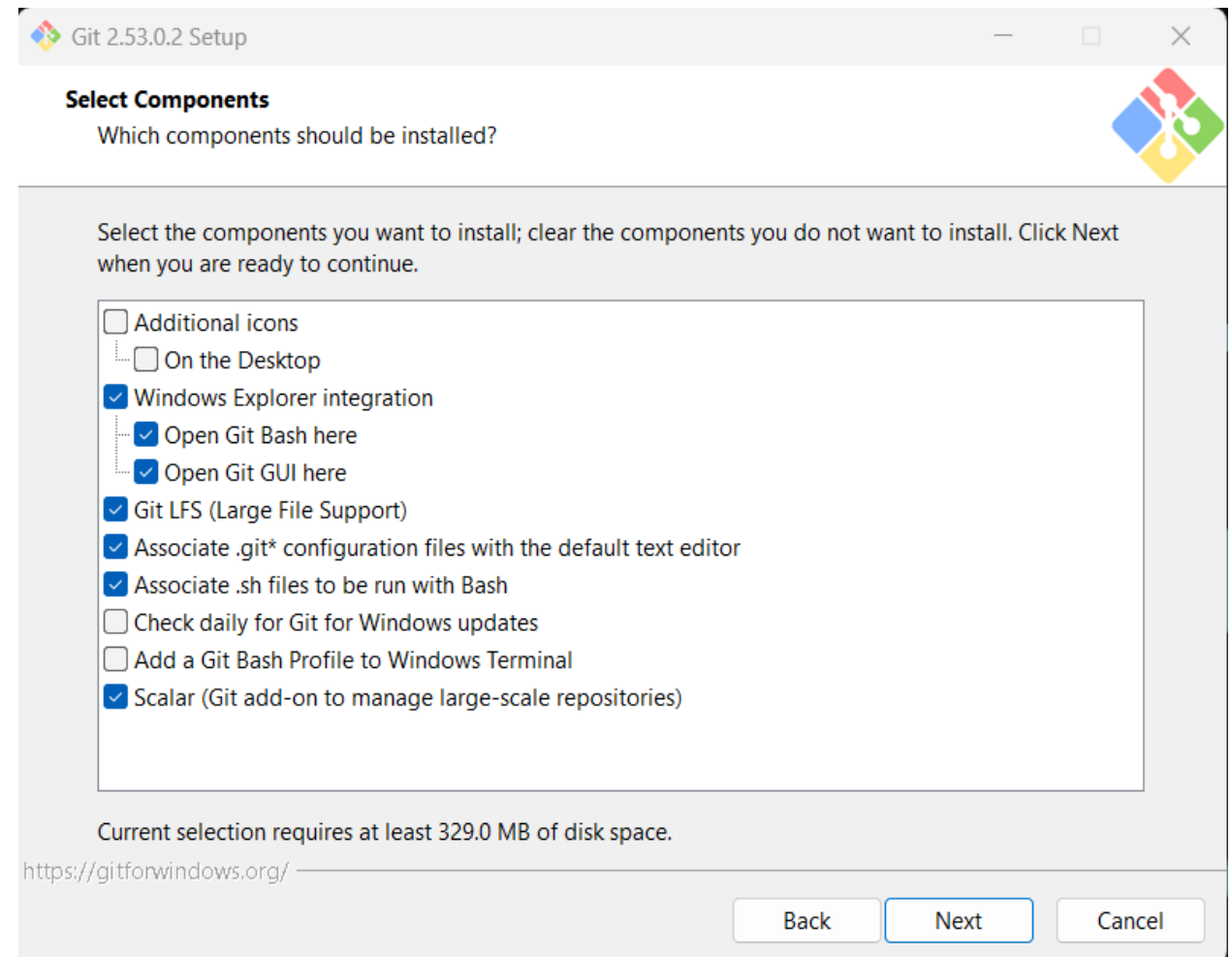
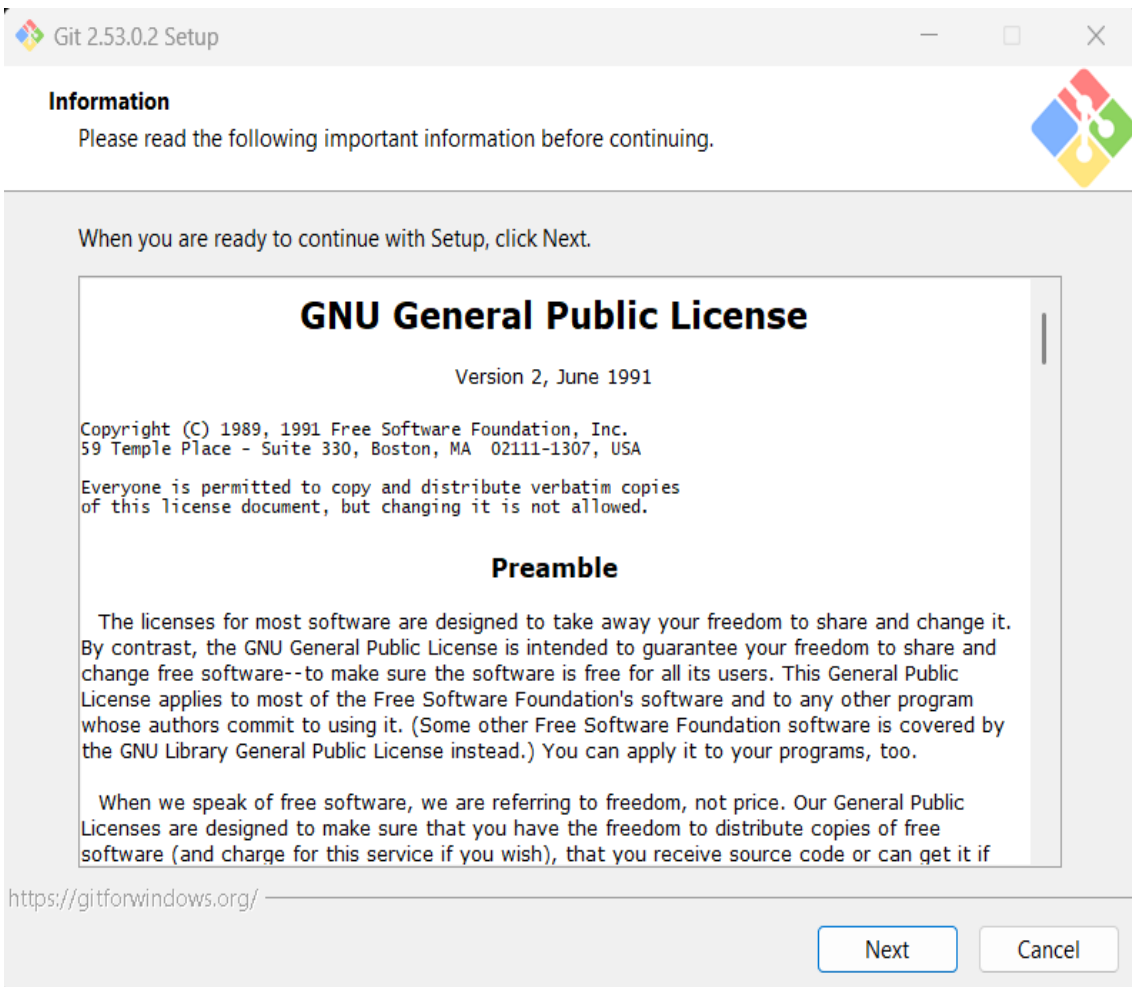
<https://git-scm.com/install/>



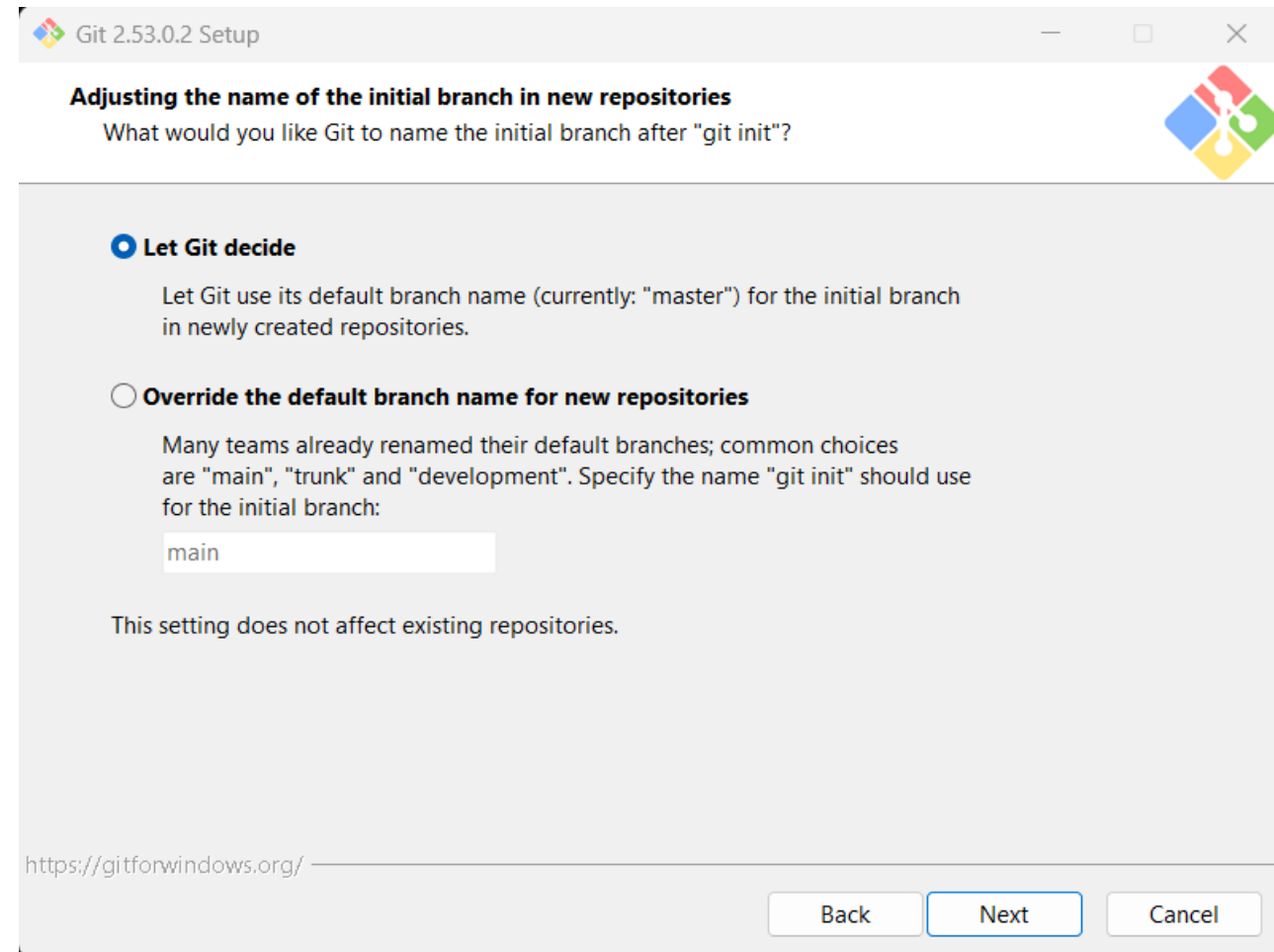
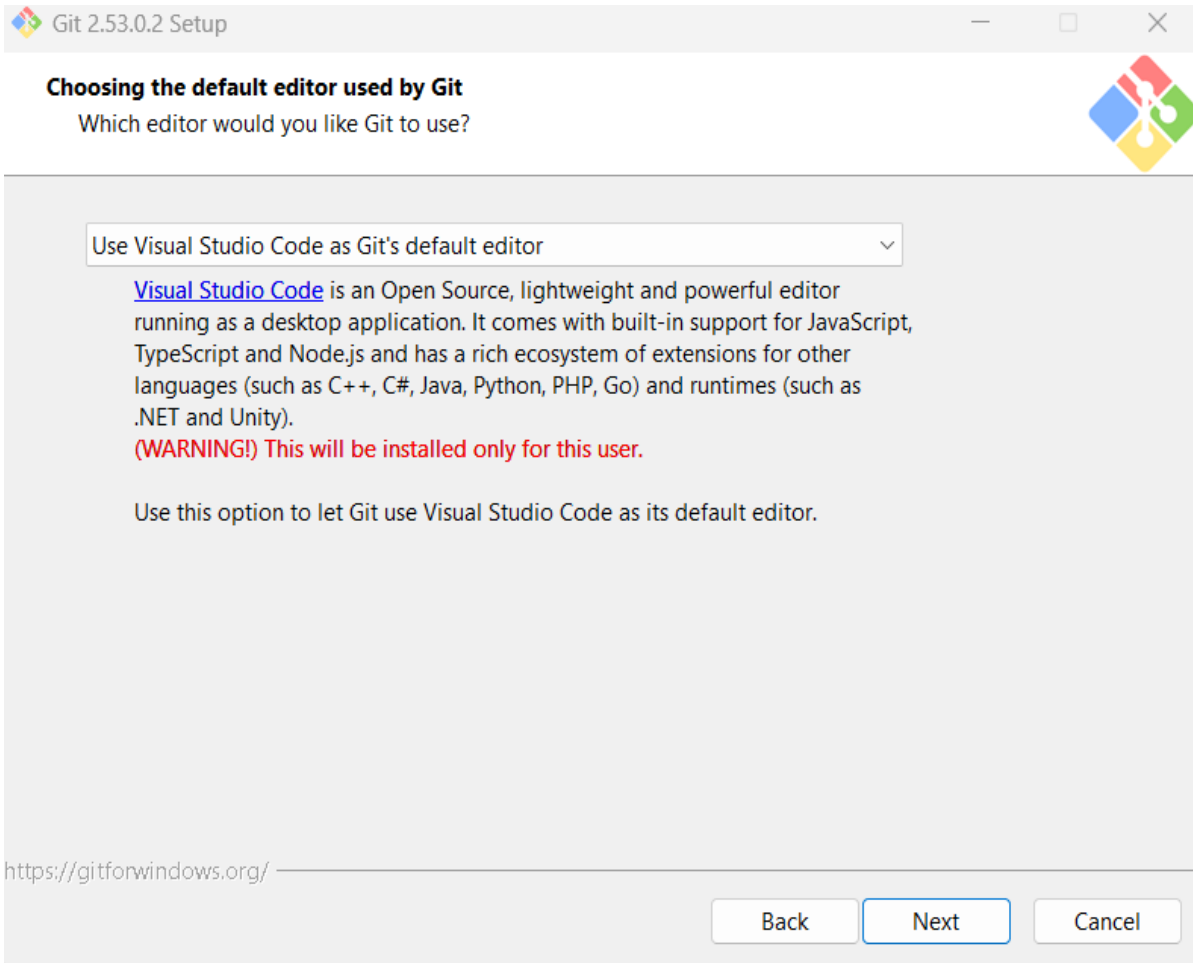
Dar doble clic en el exe descargado.



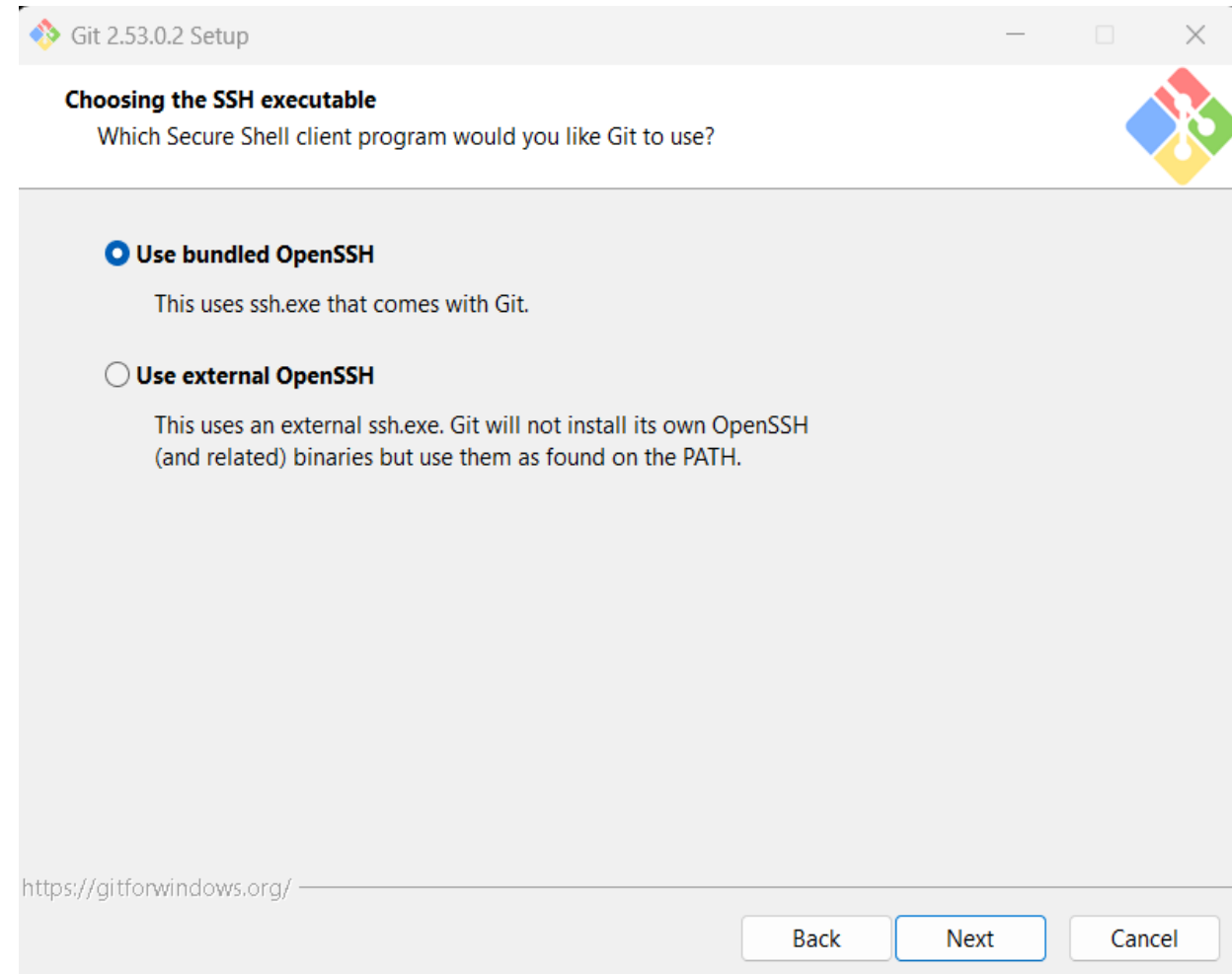
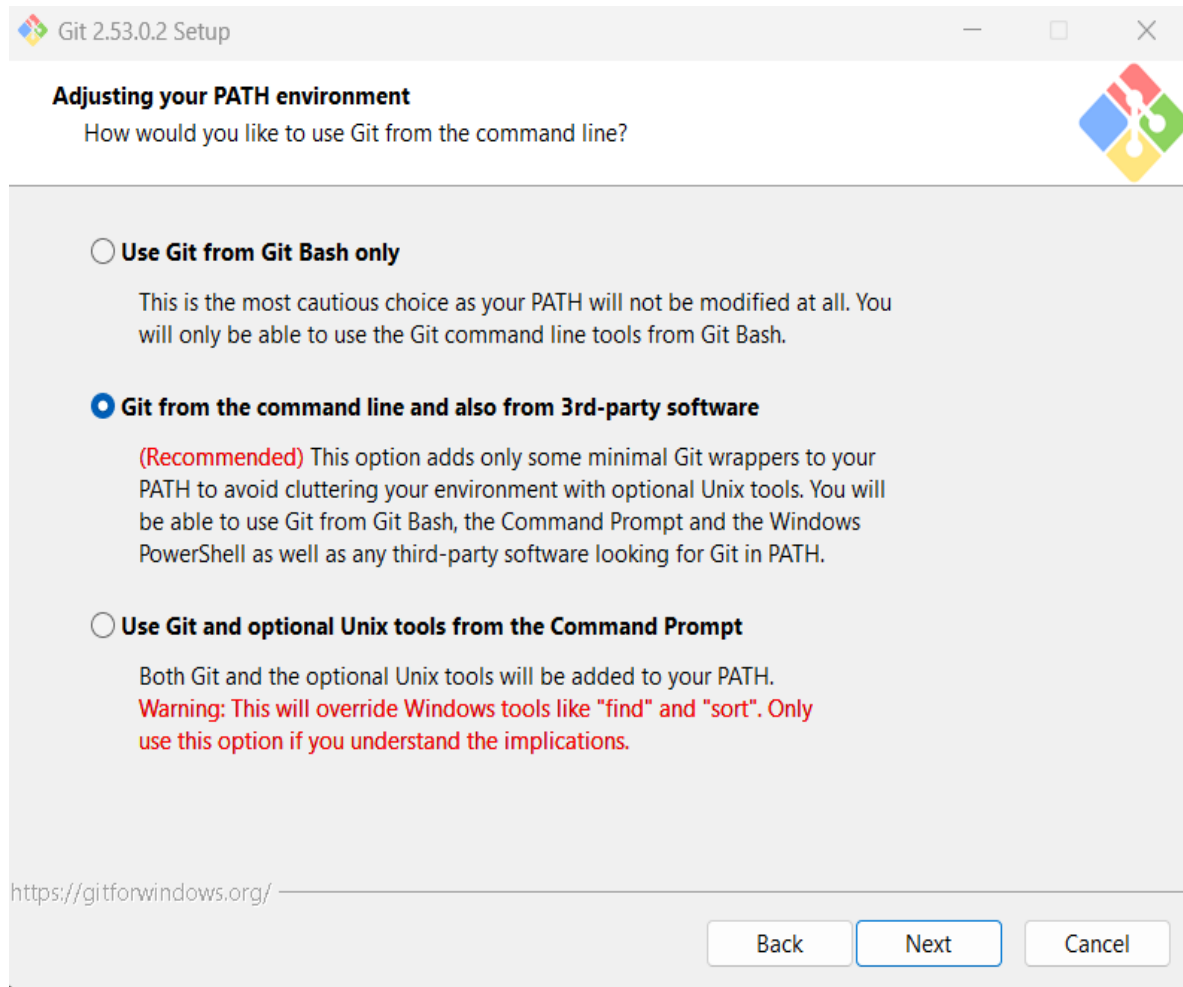
Instalación y configuración de GIT



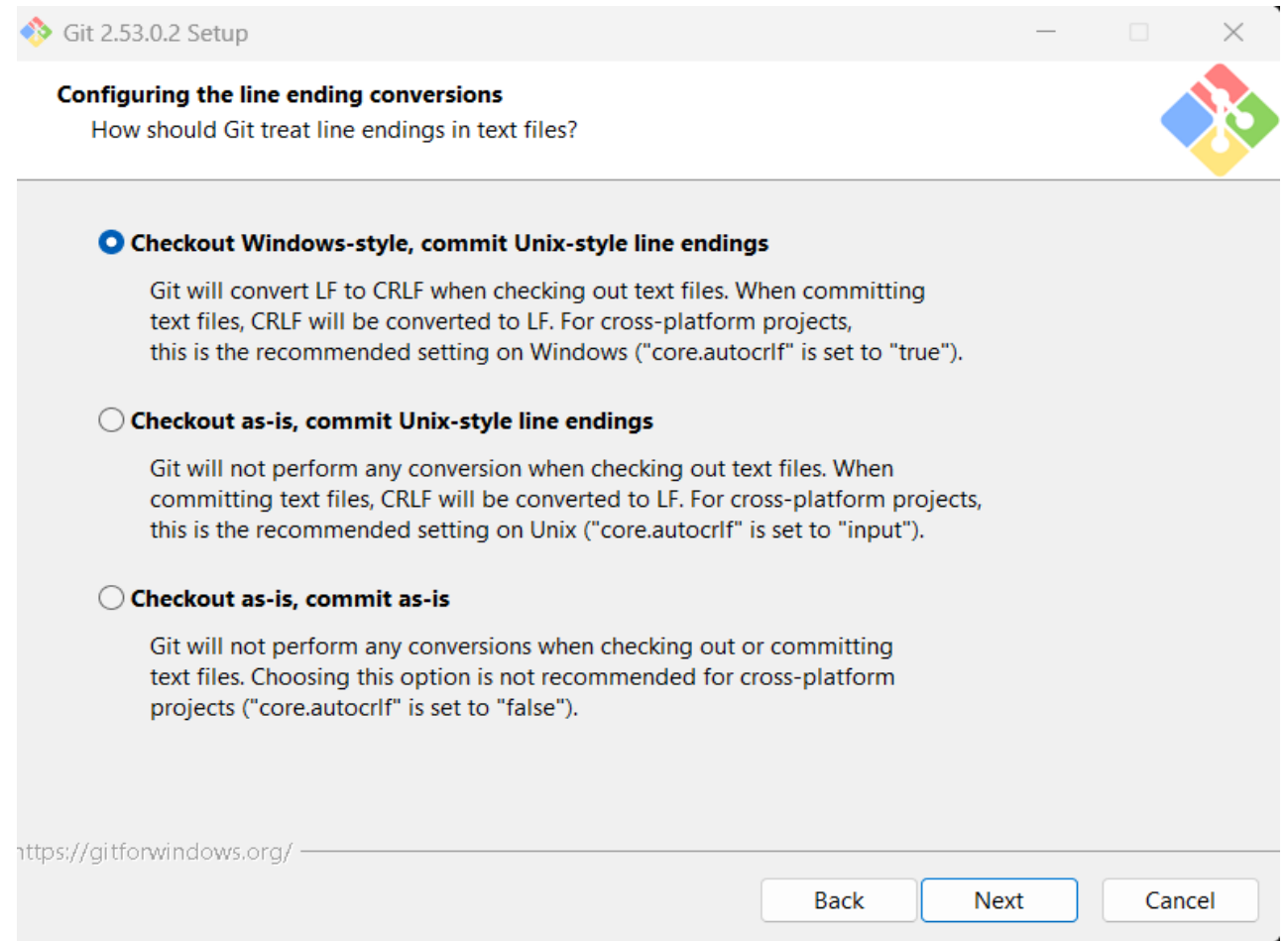
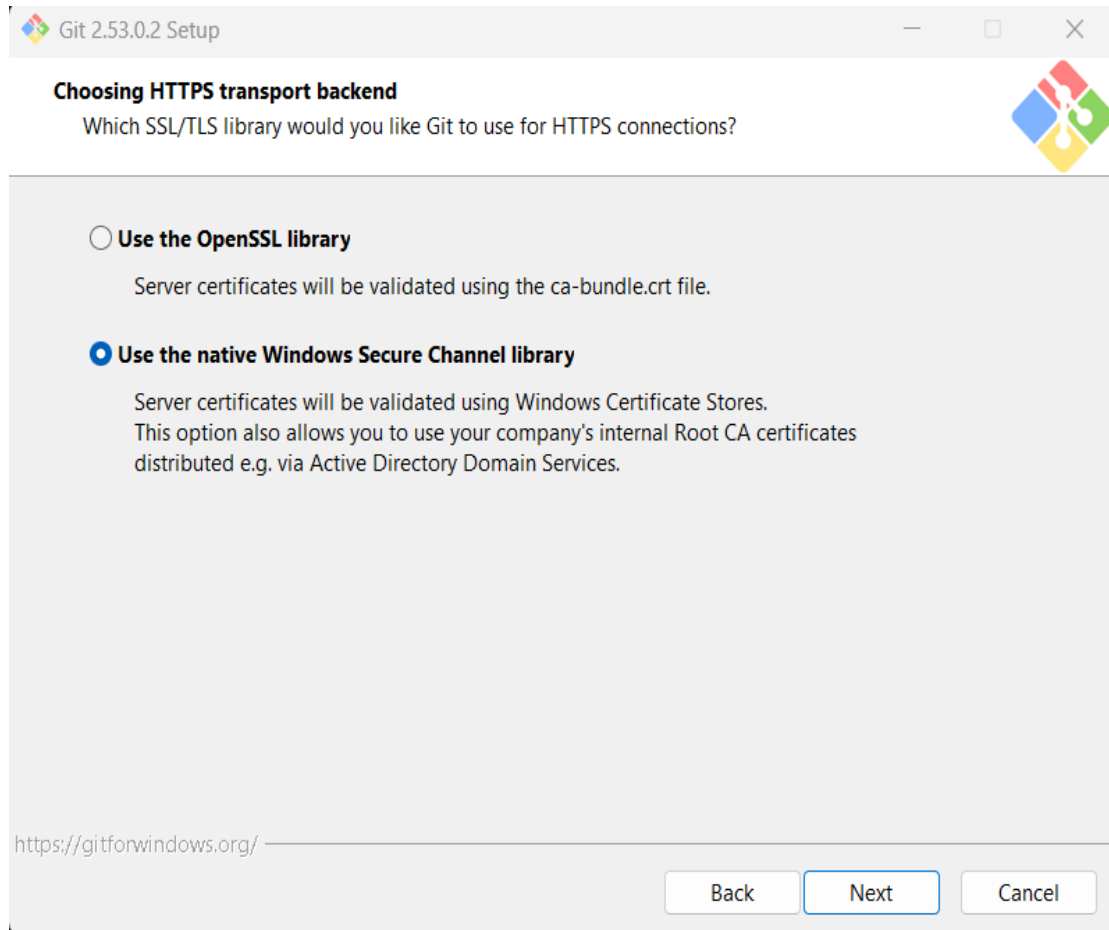
Instalación y configuración de GIT



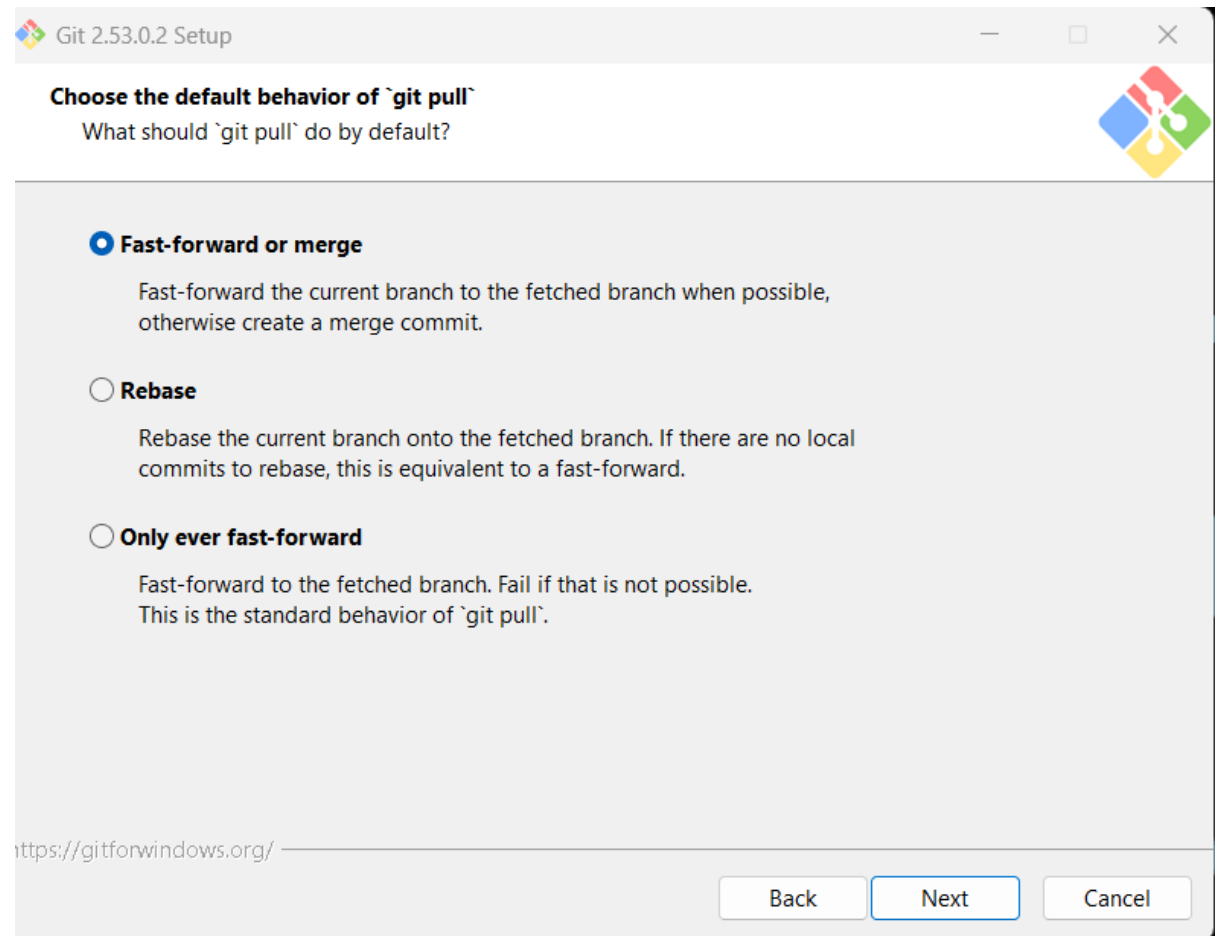
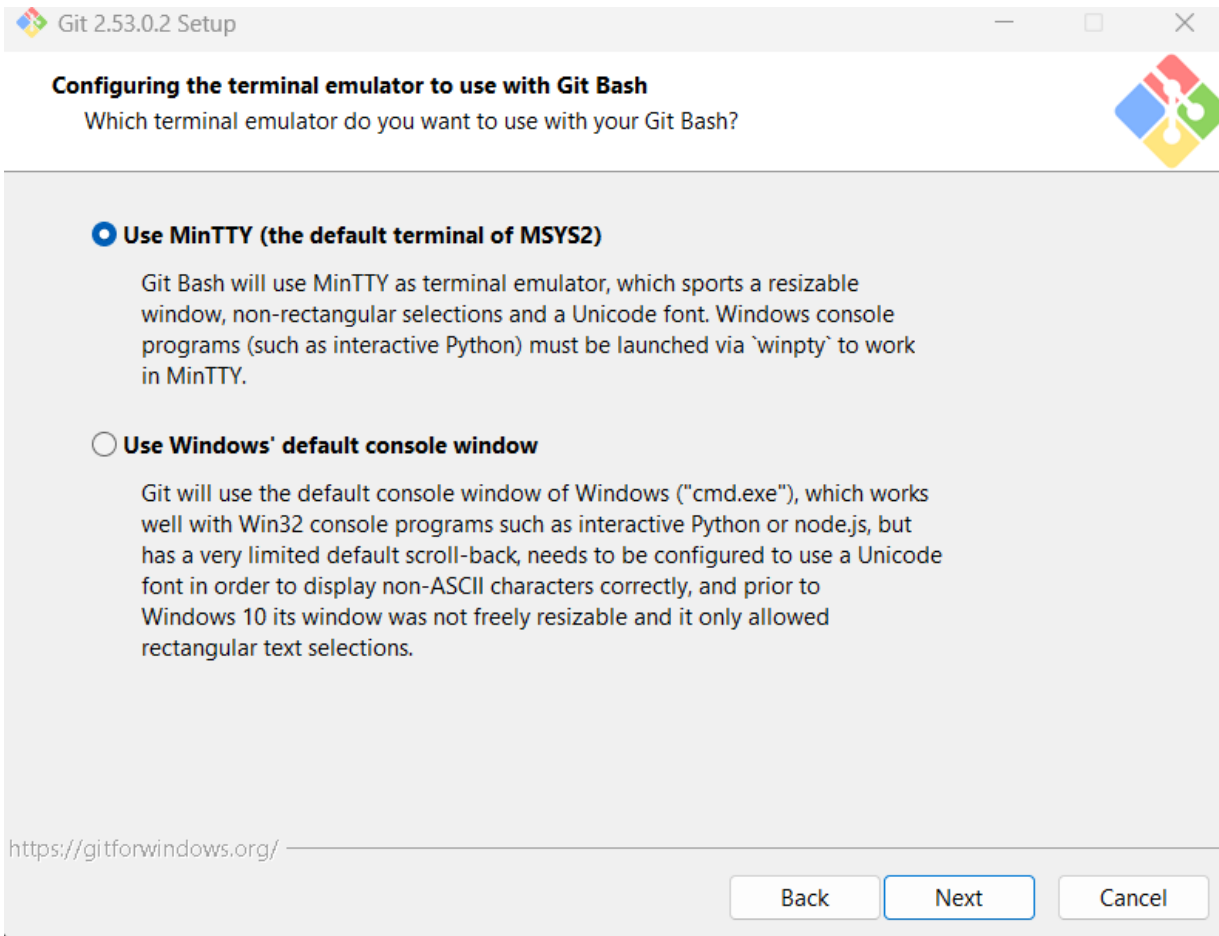
Instalación y configuración de GIT



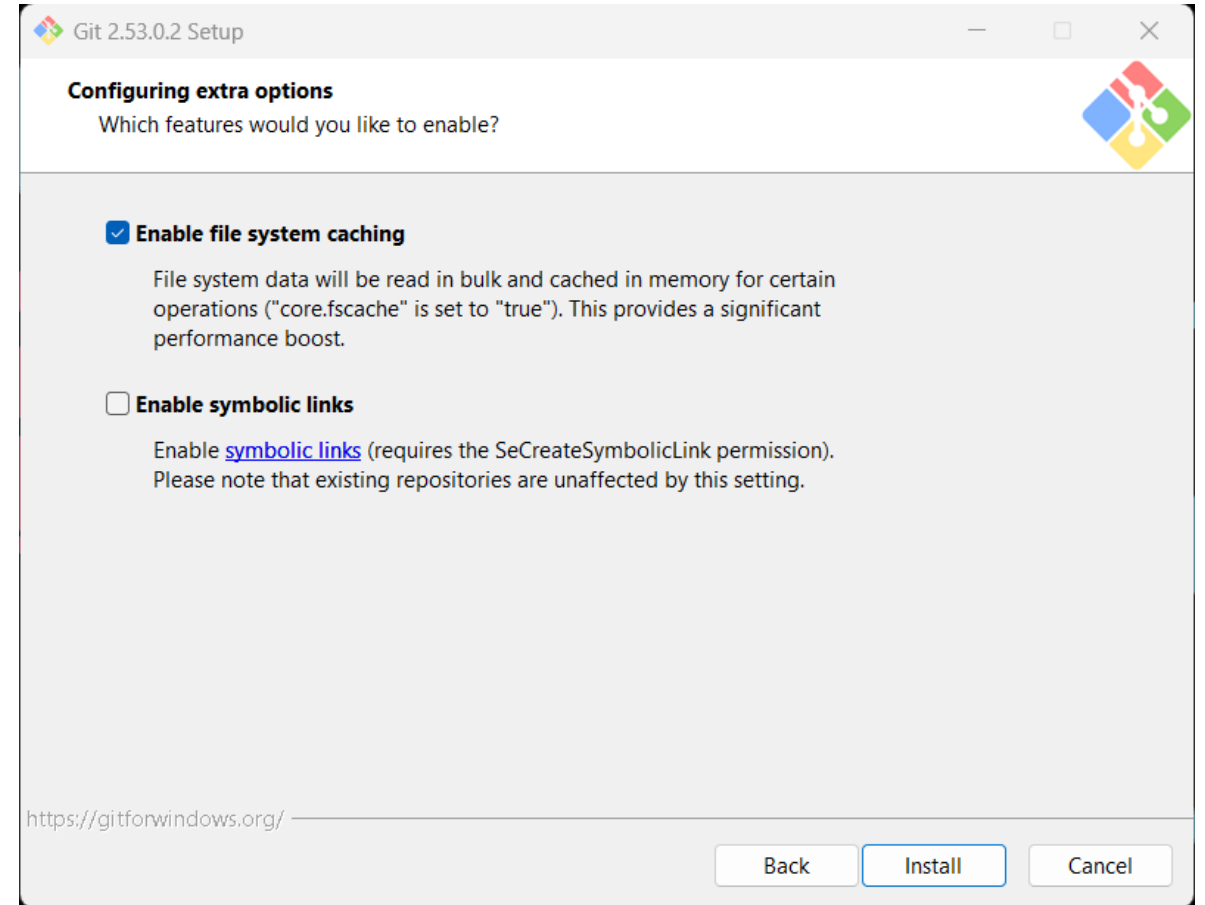
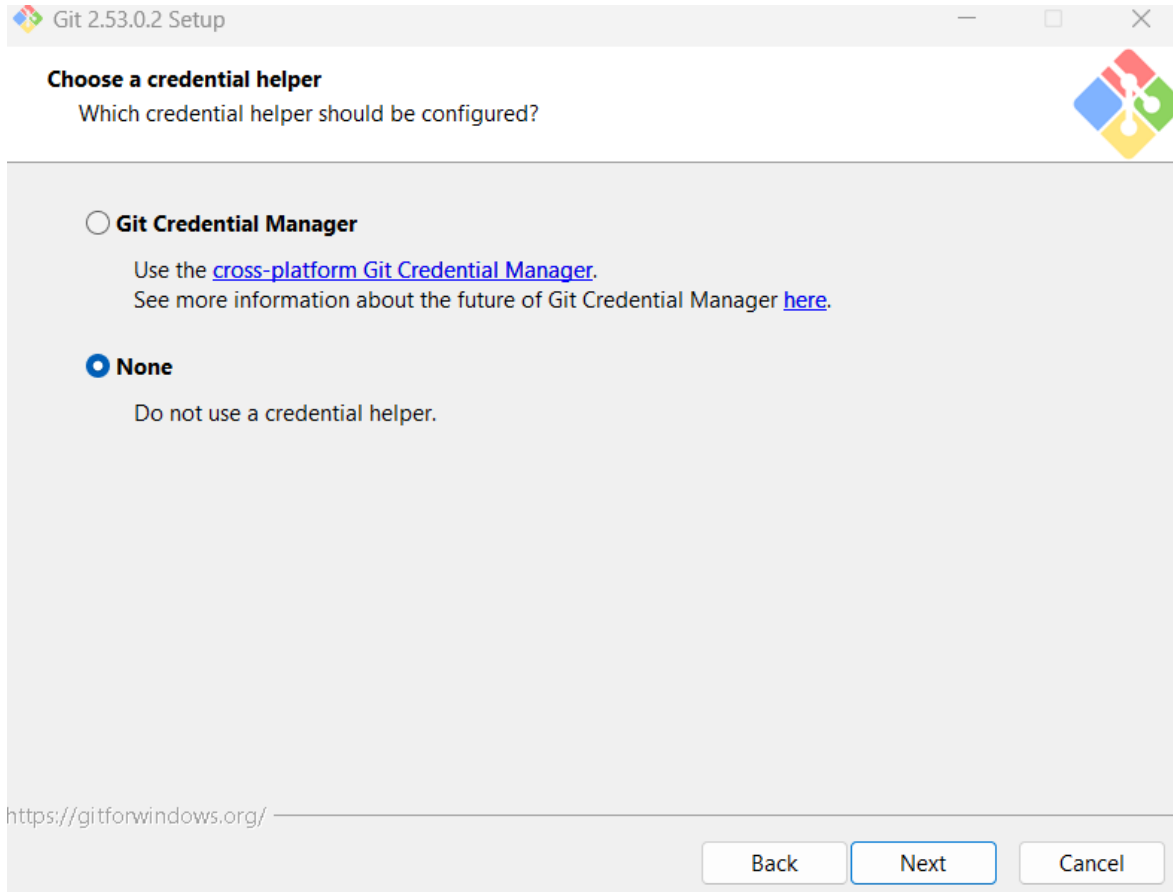
Instalación y configuración de GIT



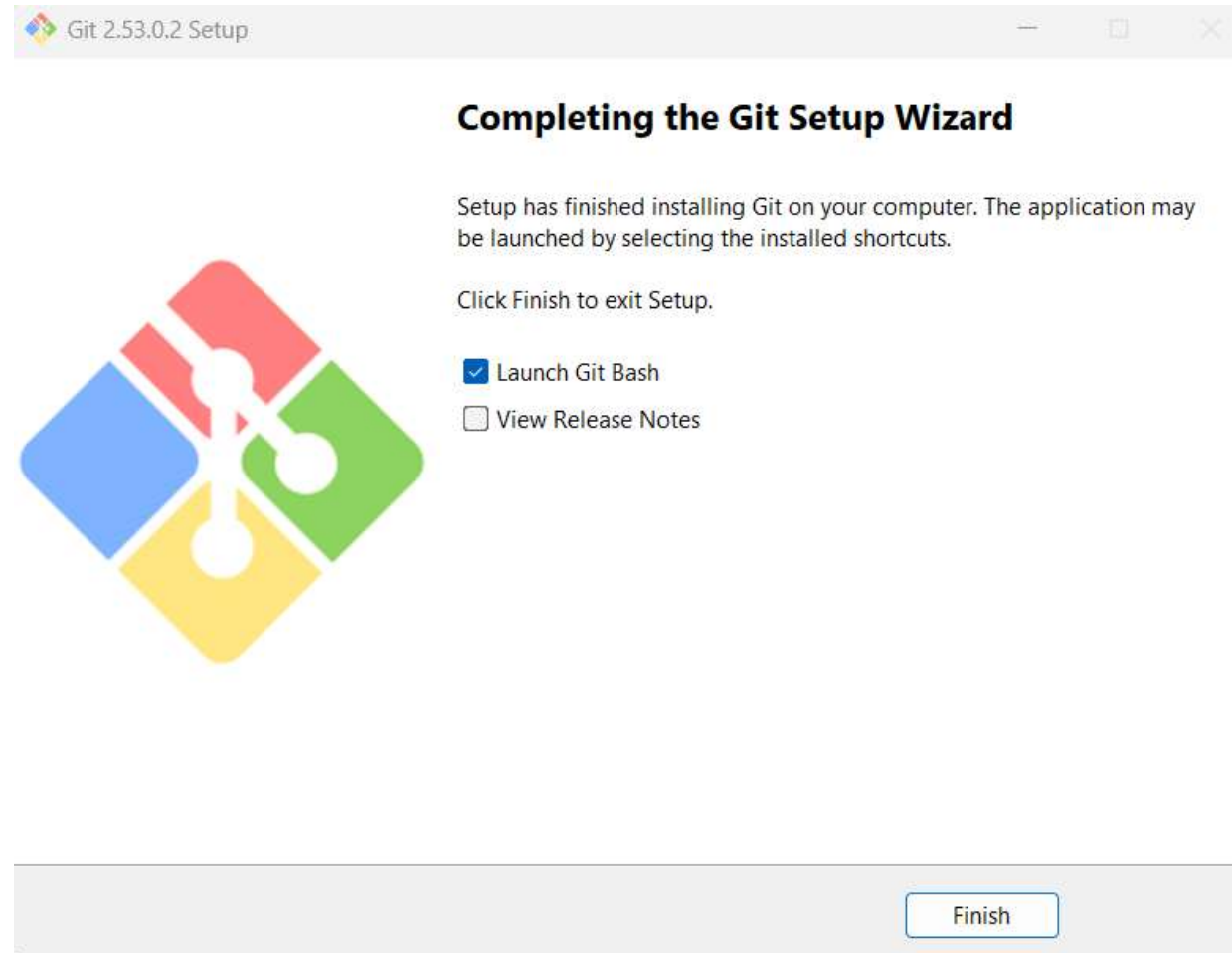
Instalación y configuración de GIT



Instalación y configuración de GIT



Instalación y configuración de GIT



GIT local (init, status, add, commit, log)

Git es un sistema de control de versiones distribuido que permite gestionar cambios localmente y sincronizarlos posteriormente. Según Chacón y Straub (2014), Git permite a los desarrolladores trabajar de forma distribuida manteniendo un historial completo del proyecto.



Flujo mental de Git local

- ➡ Working Directory: donde editas archivos. (modificaciones)
- ➡ Staging Area: donde preparas cambios. (archivos preparados para commit)
- ➡ Repository → donde guardas un commit. (commits guardados)

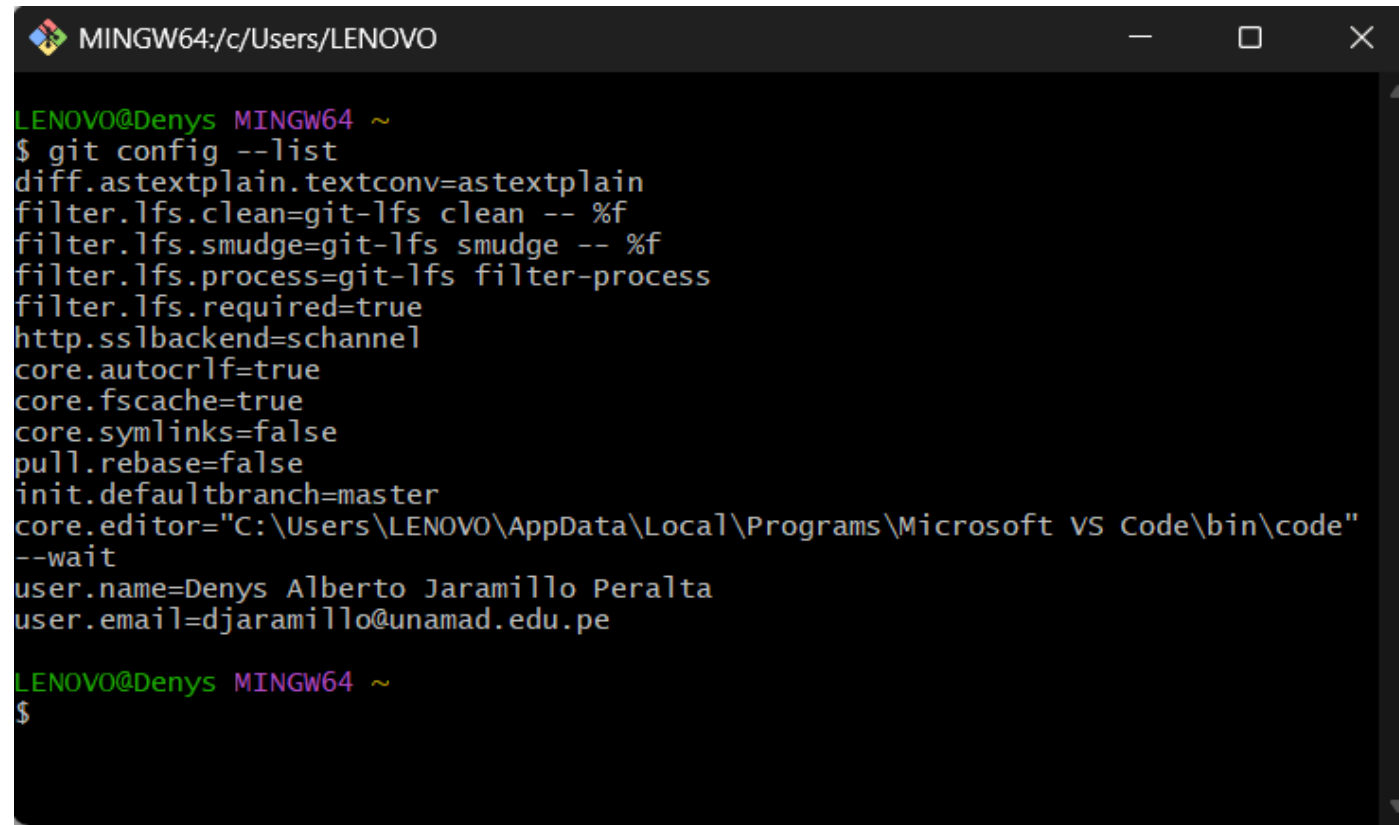
GIT local (init, status, add, commit, log)

Git configuración inicial

La configuración inicial es importante, porque nos permite configurar el nombre de usuario y el email con la cual se trabajará, el cual determinará también, el uso del repositorio remoto.

Iniciamos con el siguientes comando:

- `git config --list`
- `git config --global user.name "nombreUsuario"`
- `git config --global user.email "email"`



```
MINGW64:/c/Users/LENOVO
LENOVO@Denys MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symbols=false
pull.rebase=false
init.defaultbranch=master
core.editor="C:\Users\LENOVO\AppData\Local\Programs\Microsoft VS Code\bin\code"
--wait
user.name=Denys Alberto Jaramillo Peralta
user.email=djaramillo@unamad.edu.pe

LENOVO@Denys MINGW64 ~
$
```

GIT local (init, status, add, commit, log)

git init

Inicializa un repositorio Git en una carpeta.

Cuando inicializamos un determinado proyecto, se crea una carpeta oculta .git, el cual almacena lo siguiente: historial, objetos, referencias, ramas, metadatos.

Ejemplo

git init

```
Problems Output Debug Console Terminal Ports powershell - pruebaGit
C: > web2026-2-Pruebas > semana2 > pruebaGit
$ git init
Initialized empty Git repository in C:/web2026-2-Pruebas/semana2/pruebaGit/.git/
C: > web2026-2-Pruebas > semana2 > pruebaGit > master > ?1
11:48:31
$
```



GIT local (init, status, add, commit, log)

git status

Muestra el estado actual del repositorio.

Al momento de dar uso al comando nos mostrará los archivos no rastreados, archivos modificados y archivos listos para commit.

Ejemplo

git status

```
Problems  Output  Debug Console  Terminal  Ports  powershell - pruebaGit + v
LENOVO  C: > web2026-2-Pruebas > semana2 > pruebaGit > master > ?1
11:49:43
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.html

nothing added to commit but untracked files present (use "git add" to track)
11:49:47
$
```



GIT local (init, status, add, commit, log)

git add

Envía cambios al staging área.

Este comando se puede utilizar de dos maneras: para enviar cambios de un archivo y para enviar los cambios realizados a todo un conjunto de archivos.

Ejemplo

```
git add "nombreArchivo"  
git add .
```

git commit

Crea una fotografía lógica del proyecto.

Ejemplo

```
git commit -m "Agregar la estructura inicial del  
index.html"
```

Ojo: un commit no es guardar un archivo, sino registrar una unidad coherente de cambio.



GIT local (init, status, add, commit, log)

git log

Muestra el historial de commits.

Ejemplo

git log

Los commits constituyen la unidad fundamental del historial de cambios en los repositorios de control de versiones, y su análisis es clave para comprender mantenimiento, evolución y clasificación de cambios en proyectos de software (Le et al. 2019). Además, Git es reconocido por su conjunto de comandos flexible y amplio, lo que lo hace poderoso pero exige una enseñanza estructurada para nuevos desarrolladores (Zhang et al. 2023).



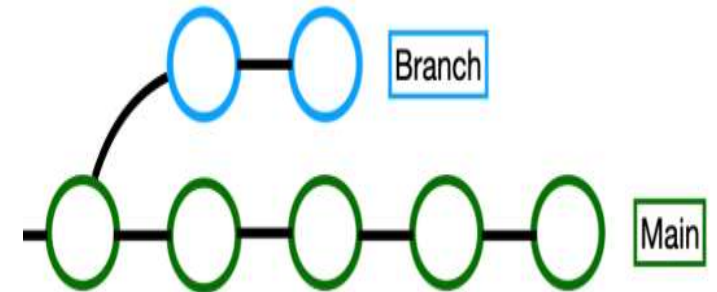
Ramas (Branch) y merge

Una rama es una línea independiente de desarrollo. Las ramas permiten trabajar en diferentes líneas de desarrollo sin afectar la principal. Según Bird et al. (2009), el uso de ramas facilita el desarrollo paralelo y reduce conflictos en proyectos colaborativos.

Idea principal:

- main: versión estable.
- feature-navbar: rama para la nueva barra de navegación.
- feature-formulario: rama para la nueva funcionalidad.

La rama es un laboratorio controlado donde puedes experimentar sin romper el producto principal.



Ramas (Branch) y merge

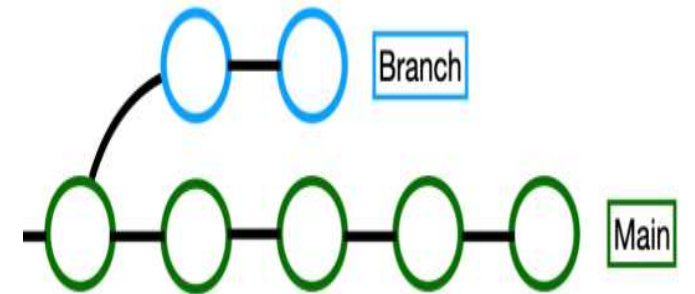
La rama es un laboratorio controlado donde puedes experimentar sin romper el producto principal.

Comandos principales

- **Crear rama:** `git branch nueva-rama`
- **Cambiar rama:** `git checkout nueva-rama`
- **Crear y Cambiar rama:** `git checkout -b nueva-rama` o `git switch feature-header` o `git switch -c feature-header`.
- **Ver ramas:** `git branch`

git merge

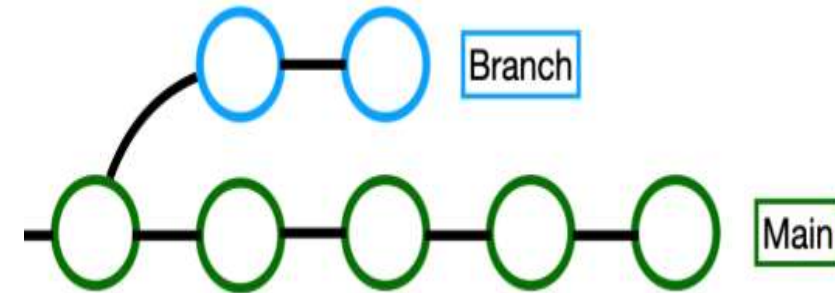
Fusiona los cambios de una rama en otra.



Ramas (Branch) y merge

Estrategias de merge

- **Fast-forward:** si la rama actual no avanzó desde que se creó la otra. Git simplemente mueve el puntero.
- **Recursive (3-way):** si ambas ramas avanzaron. Git crea un commit de merge que combina las historias.



Ejemplo:

Se cuenta con la rama principal “master”, creamos una rama: feature-header.

En la rama feature-header, actualizamos con una pequeña.

Luego regresamos a la rama principal “master” para unir la rama creada; git merge feature-header.

Para visualizar la unión de la rama con la principal, aplicamos el siguiente comando: git log –online --graph --all

Caso práctico

Dentro de la carpeta semana2, crear otra carpeta denominada landing_page, dentro de ella, crear un archivo html: index.html

Realizar el inicio del commit con la estructura principal, cuyo index tenga las siguientes características: header, section, dentro de section estará article, footer.

Crear la rama de section, con el nombre “feature-section”, en ella actualizará las características de esta sección y realice el o los commits respectivos.

Una la rama secundaria con la rama principal y al final mostrarlo mediante el comando realizado en clase.

Gracias